

Linux Administration Essentials Notes

Introduction: Beyond basic commands, Linux administration involves managing users, permissions, processes, packages, and services. These notes break down key areas with ideas for visual representation.

1 User & Group Management

- **Concept:** Controlling who can access the system and what they can do. Users (individuals) belong to Groups (collections of users).
- **User Creation:**
 - useradd <user-name>: Create a new user account.  
 - (Visual Idea: useradd command -> User icon appears)
- **User Info:**
 - id <user-name>: Displays user ID (UID), group ID (GID), and all groups the user belongs to.   
- **Groups:**
 - Groups bundle users with similar needs (e.g., devops team  x20).
 - When a user is created, a default **primary group** with the same name is usually created for them.
 - Users can belong to one **primary group** and multiple **secondary groups**.
 - (Visual Idea: User bubble connected to one 'Primary Group' bubble and multiple 'Secondary Group' bubbles)
- **Group Creation:**
 - groupadd <group-name>: Creates a new group.  
 - groupadd devops: Example: Create 'devops' group.
- **Adding Users to Groups:**
 - usermod -g <primary-group> <user>: Change a user's **primary** group.     (replaces old primary)
 - usermod -aG <secondary-group> <user>: Add a user to a **Group** (secondary), keeping their existing groups.     (adds to existing)

- `usermod -aG testers suresh`: Example: Add 'suresh' to 'testers' group.

- **Setting Password:**

- `passwd <user-name>`: Set or change a user's password. 🔑🔒

2 Permissions: Who Can Do What? 🛡️ 👁️ 📝 🏃

- **Concept:** Controlling access to files and directories. Defined for three categories: **User** (owner), **Group**, **Others**.
- **Permission Types & Values:**
 - **Read (4):** View content. 👁️ 4
 - **Write (2):** Modify, save, delete. 📝 2
 - **Execute (1):** Run a file (if a script/program) or enter a directory. 🏃 1
- **Permission String:** Seen in `ls -l` output.
 - (Visual Idea: Block showing - rwx rwx rwx structure)
- | Type | User (Owner) | Group | Others |
|----------|--------------------------|--------------------|--------------------|
| ----- | ----- | ----- | ----- |
| - (File) | rw- (Read/Write) | r-- (Read) | r-- (Read) |
| d (Dir) | rwx (Read/Write/Execute) | r-x (Read/Execute) | r-x (Read/Execute) |

 - Example: `-rw-r--r--` means file (-), owner can read/write (rw-), group can read (r--), others can read (r--).
- **Changing Permissions (chmod):** Modifies the r/w/x settings.
 - **Symbolic Mode:** Use u, g, o, a (all) and +, -, =.
 - `chmod ugo+w devops.txt`: Add write permission (+w) for **user**, **group**, and **others** on devops.txt.
 - `chmod g-w script.sh`: Remove write permission (-w) for the **group**.
 - **Numeric (Octal) Mode:** Sum up R(4), W(2), X(1) for each category (User, Group, Others).
 - `chmod 700 devops.txt`: Sets permissions for owner (4+2+1=7 -> rwx), group (0 -> ---), others (0 -> ---). (Visual Idea: Boxes showing 7 -> 4+2+1, 0, 0 mapped to rwx, ---, ---)

- **Changing Ownership (chown):** Modifies the user and group owner of a file/directory.



- chown <user>:<group> <file-name>: Change both user and group owner.
- chown <user> <file-name>: Change user owner only.
- chown :<group> <file-name>: Change group owner only.
- *Important:* File ownership can typically **only** be modified by the **root user**.



3 SSH Key-Based Access

- **Concept:** Securely logging into a remote Linux server without a password, using cryptographic keys. More secure than passwords.
- **Analogy:** Port is like a flat number in an apartment building (0-65535 total).
- SSH default port: 22. 22
- **Key Generation:**
 - ssh-keygen: Generates a public/private key pair.
 - ssh-keygen -f SurajKumar: Generate keys named 'SurajKumar' and 'SurajKumar.pub'.
- **Process Steps for Key-Based Login:** (Visual Idea: A step-by-step diagram or flowchart)

1. Create the Linux user on the server (if they don't exist). useradd SurajKumar
2. The user (SurajKumar) generates their key pair on their local machine (ssh-keygen). They keep the **private key** secret (SurajKumar) and send the **public key** (SurajKumar.pub) to the server administrator. (public)
3. The admin user on the server goes to the user's home directory (/home/SurajKumar) and creates a hidden directory named .ssh. .ssh
4. Permissions are set on .ssh so **only the owner** can access it. chmod 700 /home/SurajKumar/.ssh. (700)
5. Inside .ssh, the admin creates a file named authorized_keys. authorized_keys
6. Permissions are set on authorized_keys so **only the owner** can read/write it. chmod 600 /home/SurajKumar/.ssh/authorized_keys. (600)
7. The admin pastes the content of the user's public key (SurajKumar.pub) into the authorized_keys file.

8. Now the user (SurajKumar) should be able to log in from their local machine using their private key. ✅ 🟡

- **Login Command:**

- `ssh -i <private-key-file> <user>@<IP-address>`
 - `ssh -i ~/.ssh/SurajKumar SurajKumar@<Server-IP>`: Example login using the private key.
-

4 sudo - Granting Elevated Privileges 👑 🛡️

- **Concept:** Allowing a normal user to run commands as another user (usually root) temporarily.
 - **Configuration:** Managed via the `/etc/sudoers` file. **EDIT WITH EXTREME CAUTION** (use `visudo` command). ⚠️
 - **Common Method:** Add the user to a pre-configured group (like `wheel` or `sudo`) that has sudo privileges defined in `/etc/sudoers`.
 - `usermod -aG wheel <user-name>`: Add user to the 'wheel' group.
 - (Visual Idea: User bubble -> 'wheel' group bubble -> Arrow to 'Root User' bubble labeled 'sudo')
 - **Running Command as Root:**
 - `sudo <command>`: Execute the command with root privileges (may ask for *your* password, not root's).
-

5 Package Management 📦 🌐 ✅

- **Concept:** Standardized system for installing, updating, and removing software (packages) from repositories.
- **Role:** Connects to online repositories (like a software store 🏪), downloads packages, and handles installation/dependencies.
 - (Visual Idea: User Command -> Package Manager -> Internet Repository -> Download Package -> Install on System)
- **Common Package Managers:**
 - `apt-get / apt`: Debian/Ubuntu based systems.
 - `yum / dnf`: RHEL/CentOS/Fedora based systems.

- dnf install <package-name>: Example command to install a package.
 - **Repositories:** Configured in files, often under /etc/yum.repos.d/ (for yum/dnf) or /etc/apt/sources.list.d/ (for apt).
-

6 Service Management (systemctl) ⚙️ 🟢 🔴 ? 🔄

- **Concept:** Managing system services or daemons (background processes) like web servers (nginx), databases, etc.
 - **systemctl Commands:**
 - systemctl start <service>: Begin a service. 🟢
 - systemctl stop <service>: Halt a service. 🔴
 - systemctl status <service>: Check if a service is running, active state, and recent logs. ? ✅ ❌
 - systemctl restart <service>: Stop and then start a service. 🔄
 - systemctl enable <service>: Configure a service to start automatically on boot. 💾 boot
 - systemctl disable <service>: Configure a service *not* to start automatically on boot. 🚫 boot
 - **Example:** Install and start nginx web server.
1. Install: dnf install nginx or apt-get install nginx 📦
 2. Start: systemctl start nginx 🟢
 3. Check status: systemctl status nginx ?
 4. Enable on boot: systemctl enable nginx 💾
-

7 Process Management 🧠 🏃 🔴 📊

- **Concept:** Understanding and managing running programs (processes) handled by the Linux kernel. (Analogy: Like tasks assigned to teams/managers - TM->TL->...)
- **Process ID (PID):** A unique number assigned to each running process. 📄
12
34
- **Parent Process ID (PPID):** The PID of the process that started the current process.
🌲 (PID -> PPID)
- **Viewing Processes (ps):** Displays current running processes.
 - ps: Basic view (your processes).

- `ps -ef`: Full listing of processes on the system (User, PID, PPID, CPU usage, Start Time, TTY, TIME, Command). (Visual Idea: Show sample output with columns labeled).
 - `ps -ef | grep <process-name>`: Find a specific process by name using piping. 🔍
 - **Foreground vs. Background:**
 - Foreground: Command runs and ties up your terminal until finished.
 - Background: Command runs without tying up terminal. Use `&` at the end of the command. `mycommand &` 🏃 ➡ 🌄
 - **Terminating Processes (kill):** Send signals to processes to stop them.
 - `kill <PID>`: Send a default signal (TERM) requesting the process to terminate gracefully. 👤 🔴
 - `kill -9 <PID>`: Send signal 9 (KILL) forcefully terminates the process. Cannot be ignored. 💥 🔴 (Use only if `kill <PID>` doesn't work).
 - **Monitoring Resources (top):** View processes and system resource usage (CPU, RAM).
 - `top`: Show interactive list of top resource-consuming processes. 📊 📈
 - `top -p <PID>`: Monitor resources for a **specific process** by its PID. 📊 🧑
-

8 Network Management: Open Ports 🌐 🧑 🛠

- **Concept:** Checking which ports are open and listening for connections on the server.
 - **Command:**
 - `netstat -lntp`: Displays listening sockets, **numeric addresses**, **tCP ports**, **process IDs**.
 - (Visual Idea: List of IP:Port -> State (LISTEN) -> PID/Program Name)
 - Shows you which service is listening on which IP address and port.
-

9 Troubleshooting Flow (Example: Service Down) 🧑 🛠

- (Visual Idea: A simple linear or branching flowchart)
 1. **Check Service Status:** `systemctl status <service-name>` (Is it active? Why did it fail?) ? ❌ ✅
 2. **Check Process Status:** `ps -ef | grep <service-name>` (Is the process actually running? Multiple instances?) 🏃 🔍

3. **Check Listening Port:** `netstat -lntp` (Is the service listening on the expected port?) 🌐 🔌 💡
 4. **Check Resource Usage:** `top -p <PID>` (Is the process consuming too much CPU/RAM, causing issues?) 📊
-

Conclusion: Mastering these administrative tasks is fundamental to effectively managing Linux servers and infrastructure in a DevOps environment! Keep exploring and practicing!



SURAJ KUMAR PADHI.